Assignment – 13.5

Name:B.shashank
H.no: 2303A51431
Batch: 21

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the code.
• Instructions:
o Identify global variables used across functions.
o Refactor the code to pass values using function parameters.
o Improve modularity and testability.
• Sample Legacy Code:

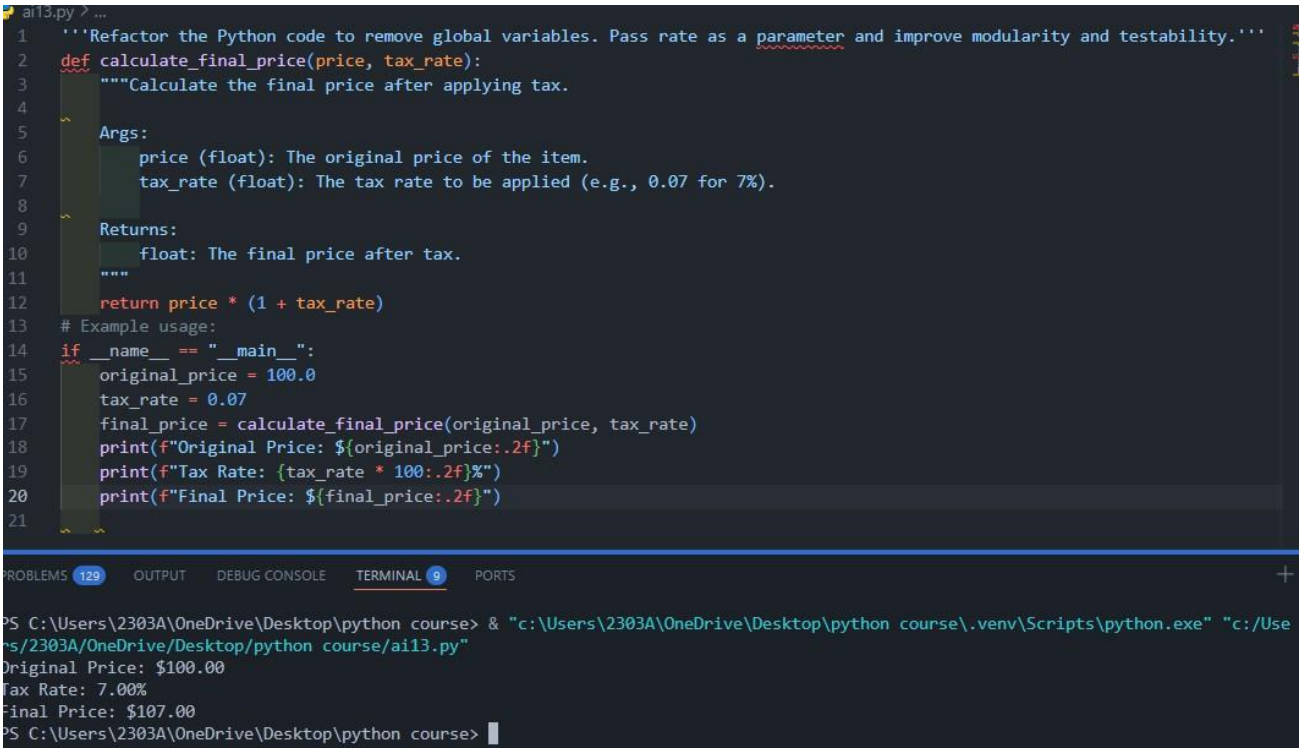
```
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
```

• Expected Output:
o Refactored version passing rate as a parameter or using a configuration structure.

Prompt:

Refactor the Python code to remove global variables. Pass rate as a parameter and improve modularity and testability.

Code:



```
ai13.py > ...
'''Refactor the Python code to remove global variables. Pass rate as a parameter and improve modularity and testability.'''
def calculate_final_price(price, tax_rate):
    """Calculate the final price after applying tax.

    Args:
        price (float): The original price of the item.
        tax_rate (float): The tax rate to be applied (e.g., 0.07 for 7%).

    Returns:
        float: The final price after tax.
    """
    return price * (1 + tax_rate)
# Example usage:
if __name__ == "__main__":
    original_price = 100.0
    tax_rate = 0.07
    final_price = calculate_final_price(original_price, tax_rate)
    print(f"Original Price: ${original_price:.2f}")
    print(f"Tax Rate: {tax_rate * 100:.2f}%")
    print(f"Final Price: ${final_price:.2f}")
```

```
PROBLEMS 129  OUTPUT  DEBUG CONSOLE  TERMINAL 9  PORTS

PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Use
rs/2303A/OneDrive/Desktop/python course/ai13.py"
Original Price: $100.00
Tax Rate: 7.00%
Final Price: $107.00
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

- Global variable removed.
- Function is now reusable and testable.
- Follows modular programming principles.

Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.
• Focus Areas:
o Readability
o Logical simplification
o Maintainability
Legacy Code:

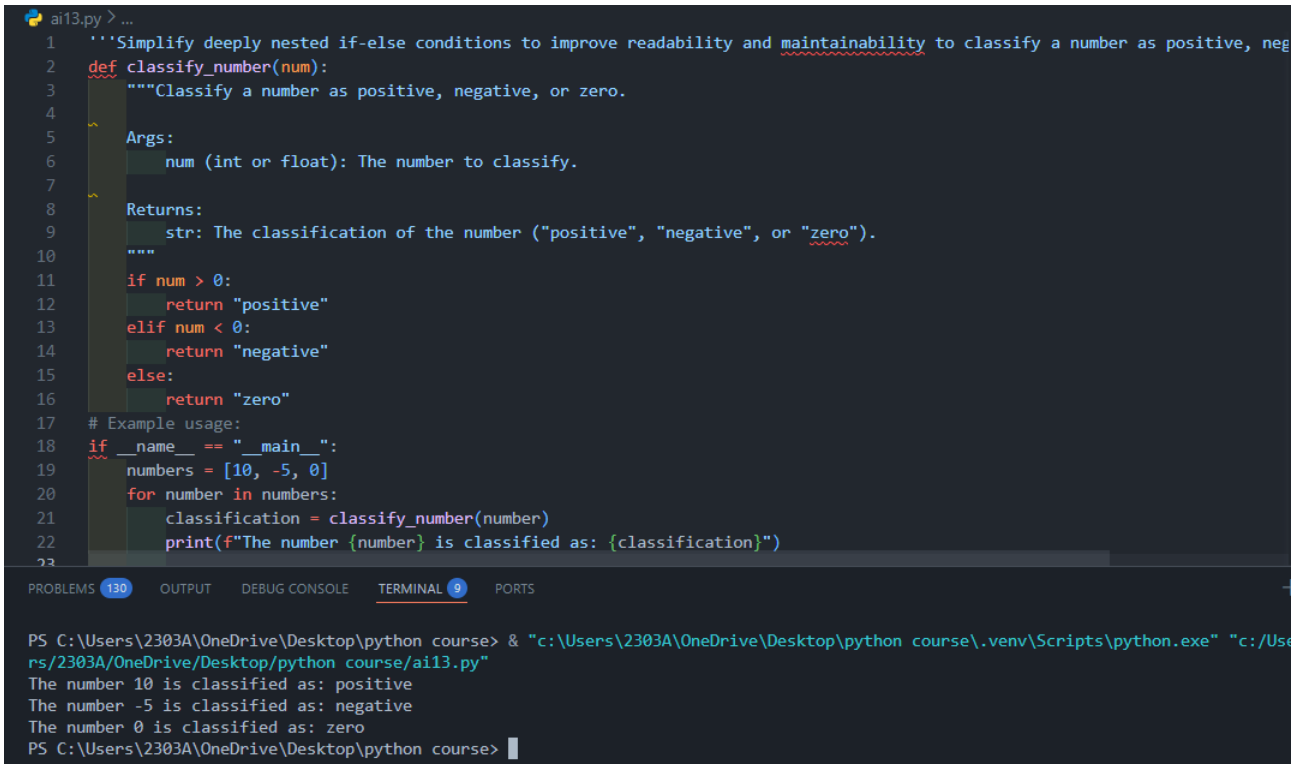
```
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
print("Very Good")
```

else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
Expected Outcome:
o Flattened logic using guard clauses or a mapping-based
approach.

Prompt:

Simplify deeply nested if-else conditions to improve readability and maintainability.

Code:



Observation:

- Removed nested structure.
- Improved readability.
- Easier to modify grade ranges.

Task 3 (Refactoring Repeated File Handling Code)

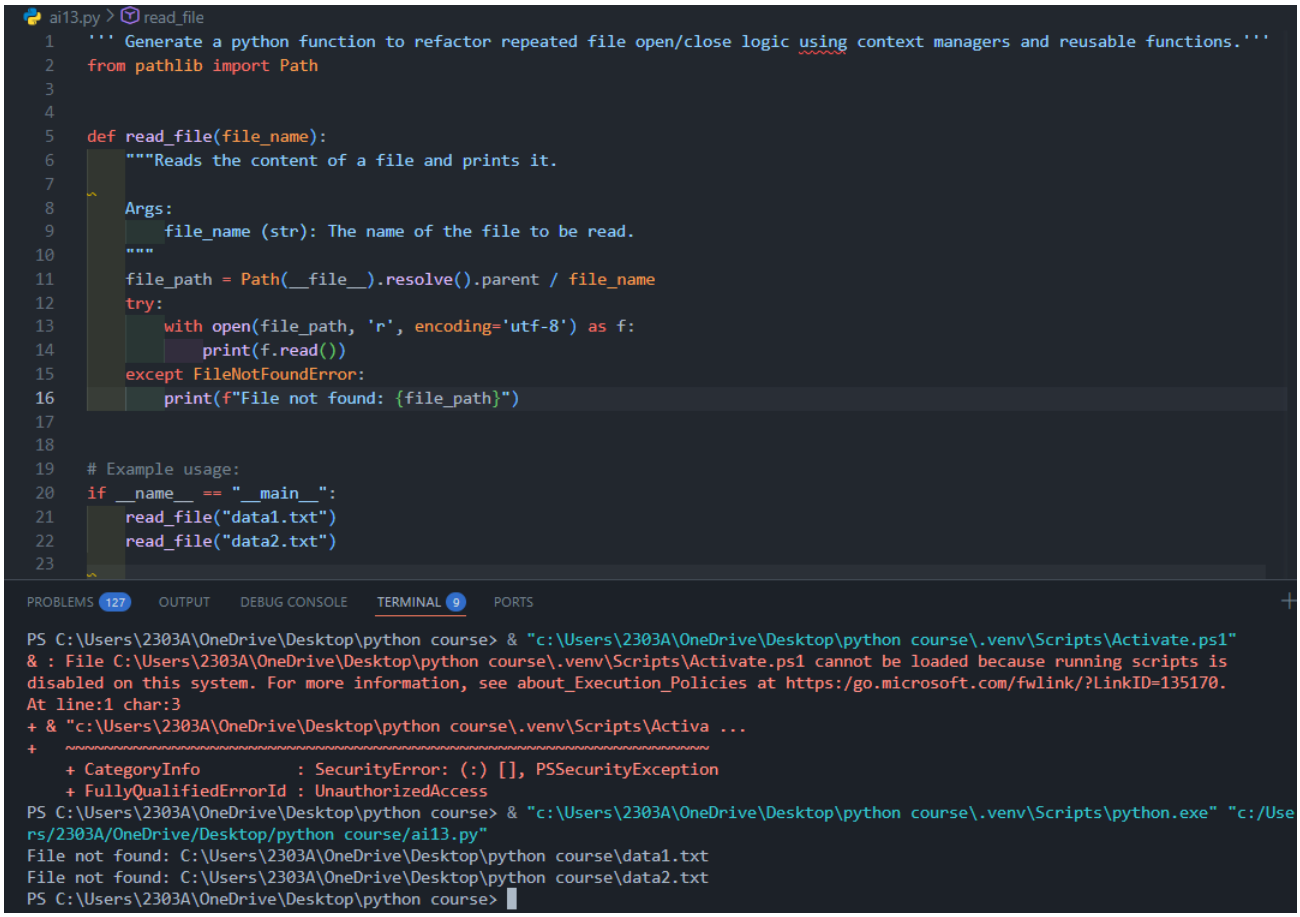
• Task: Use AI to refactor repeated file open/read/close logic.
• Focus Areas:
o DRY principle
o Context managers

o Function reuse
Legacy Code:
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
Expected Outcome:
o Reusable function using with open() and parameters.

Prompt:

Refactor repeated file open/close logic using context managers and reusable functions.

Code:



Observation:

- Applied DRY principle.
- Used with open() (context manager).
- Prevents file leaks.

Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied")

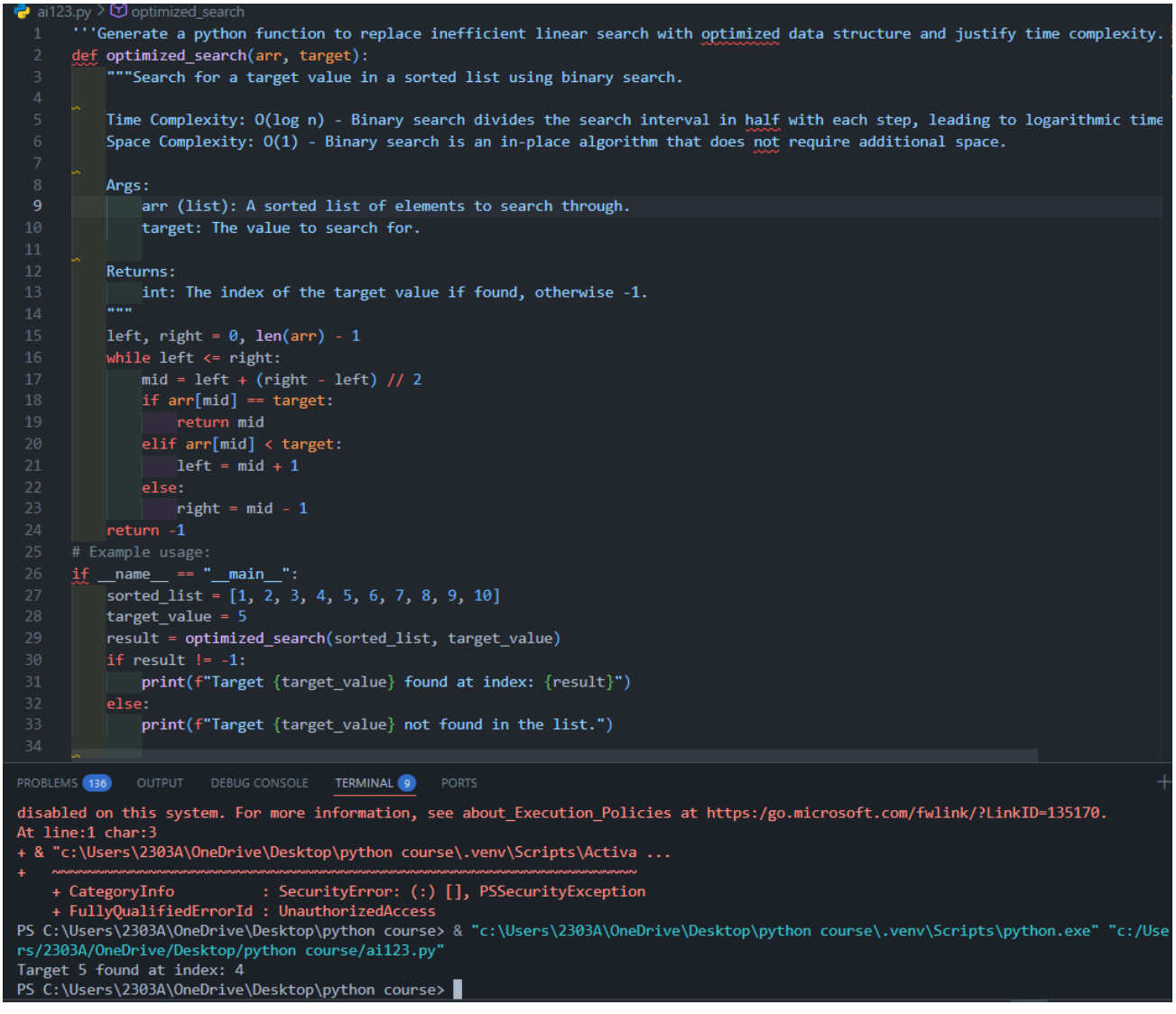
Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Prompt:

Replace inefficient linear search with optimized data structure and justify time complexity.

Code:



Observation:

- Changed list → set.
- Time complexity improved from **O(n)** to **O(1)** average case.
- More scalable for large datasets.

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

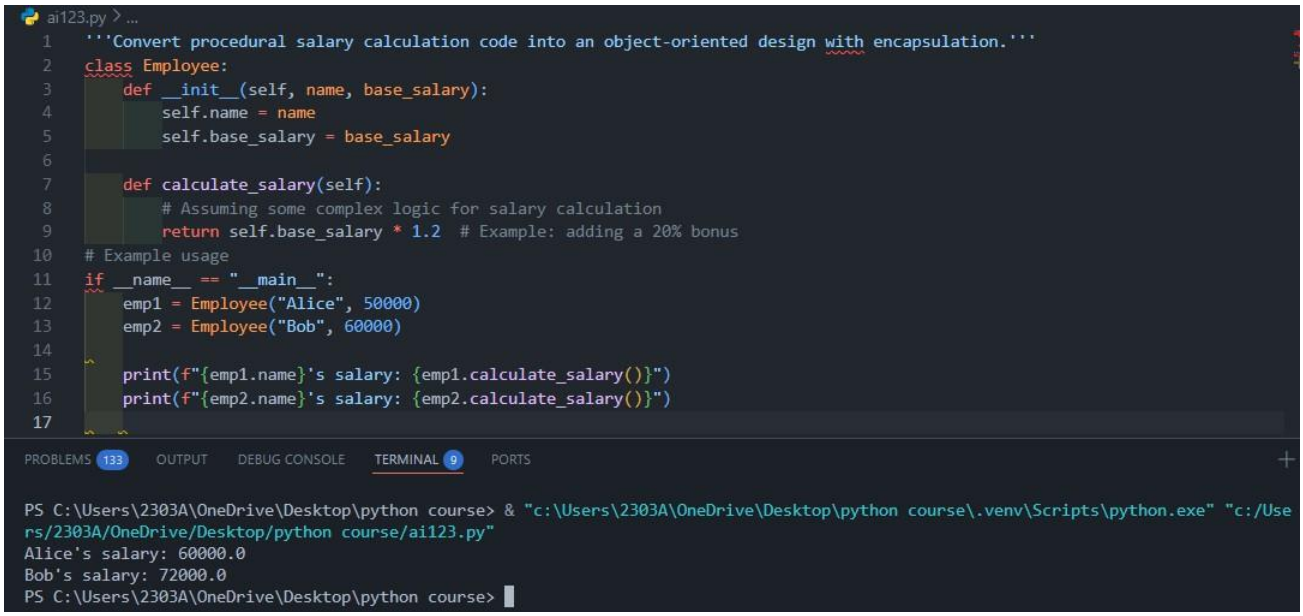
o Encapsulation

Legacy Code:
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
Expected Outcome:
o A class like EmployeeSalaryCalculator with methods and
attributes.

Prompt:

Convert procedural salary calculation code into an object-oriented design with encapsulation.

Code:



Observation:

- Applied OOP principles.
- Encapsulation achieved.
- Reusable for multiple employees.

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

```
total = 0
for i in range(1, 1000000):
if i % 2 == 0:
total += i
print(total)
```

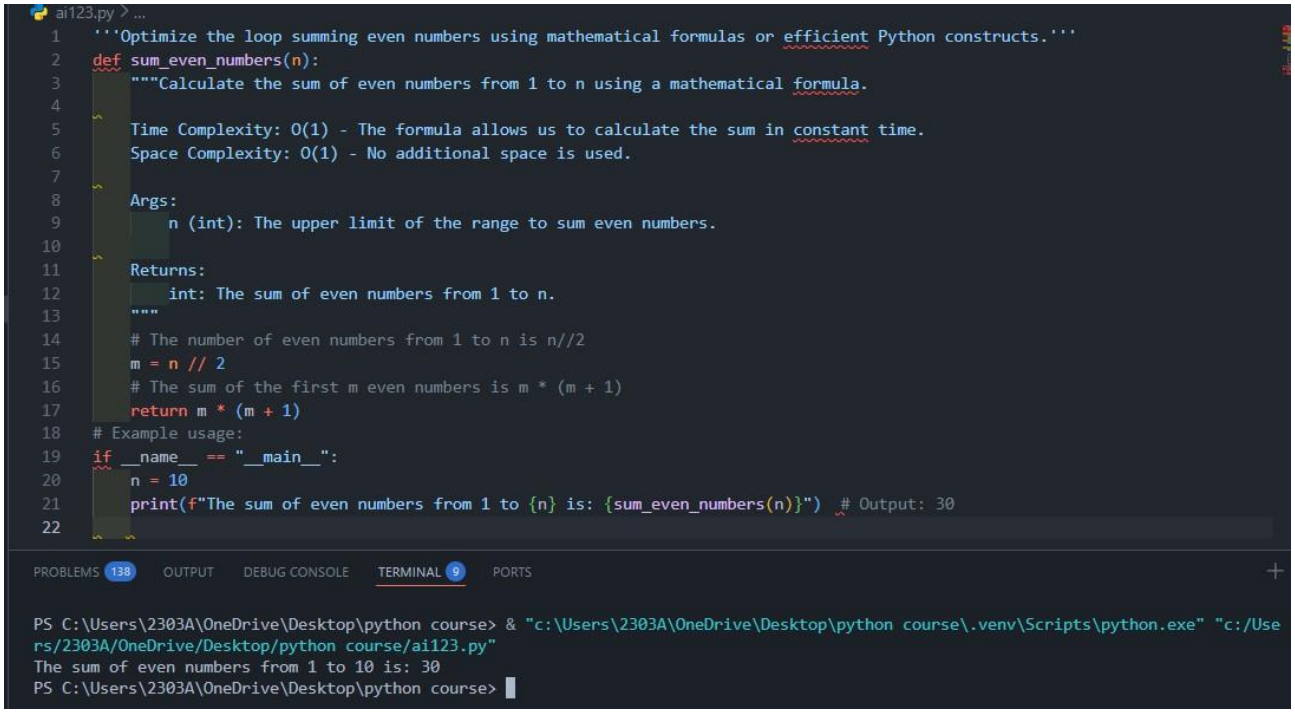
Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Prompt:

Optimize the loop summing even numbers using mathematical formulas or efficient Python constructs.

Code:



Observation:

- Replaced O(n) loop with O(1) formula.
- Massive performance improvement.
- Suitable for large-scale computations.

Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

```
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)
print(data)
```

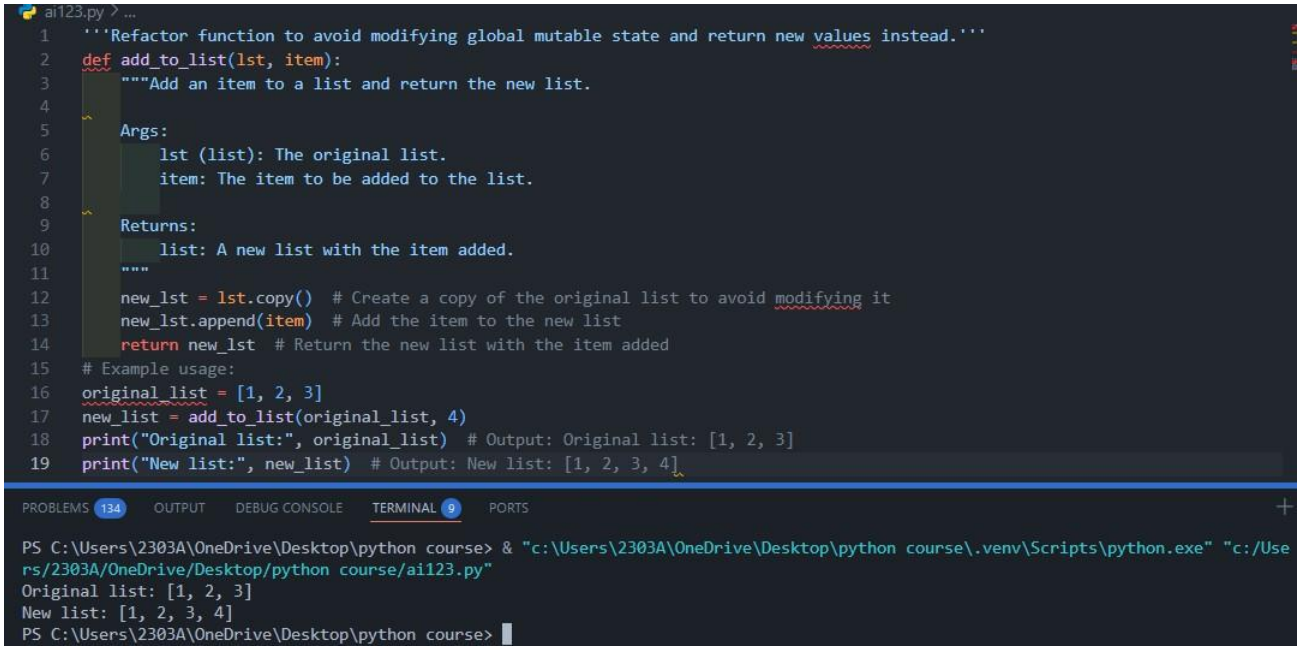
Expected Outcome:

o Refactored function returning values instead of mutating globals.

Prompt:

Refactor function to avoid modifying global mutable state and return new values instead.

Code:



Observation:

- Removed global mutation.
- Function is now pure and predictable.
- Improves testability.

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
if any(c.isdigit() for c in password):
if any(c.isupper() for c in password):
print("Valid Password")
else:
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
```

Expected Outcome:

o Separate validation functions with clear responsibility

Prompt:

Modularize password validation rules into separate functions to improve readability and testability.

Code:

ai123.py > ...

```python
'''Modularize password validation rules into separate functions to improve readability and testability'''
def is_valid_length(password):
    return len(password) >= 8
def has_uppercase(password):
    return any(char.isupper() for char in password)
def has_lowercase(password):
    return any(char.islower() for char in password)
def has_digit(password):
    return any(char.isdigit() for char in password)
def has_special_char(password):
    special_characters = "!@#$%^&*()-_=+[]{}|;:'\",.<>?/"
    return any(char in special_characters for char in password)
def validate_password(password):
    """Validate the password based on defined rules.

    Args:
        password (str): The password to validate.

    Returns:
        bool: True if the password is valid, False otherwise.
    """
    if not is_valid_length(password):
        print("Password must be at least 8 characters long.")
        return False
    if not has_uppercase(password):
        print("Password must contain at least one uppercase letter.")
        return False
    if not has_lowercase(password):
        print("Password must contain at least one lowercase letter.")
        return False
    if not has_digit(password):
        print("Password must contain at least one digit.")
        return False
    if not has_special_char(password):
        print("Password must contain at least one special character.")
        return False
    return True
# Example usage
if __name__ == "__main__":
    password = input("Enter a password to validate: ")
    if validate_password(password):
```

PROBLEMS 139 OUTPUT DEBUG CONSOLE TERMINAL 9 PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Users/2303A/OneDrive/Desktop/python course/ai1
"
Enter a password to validate: Kushal1234
Password must contain at least one special character.
Password is invalid.
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Users/2303A/OneDrive/Desktop/python course/ai1
"
Enter a password to validate: Kushal@1234
Password is valid.
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

- Broke validation into small reusable functions.
- Improved readability.
- Easier to unit test each rule independently.